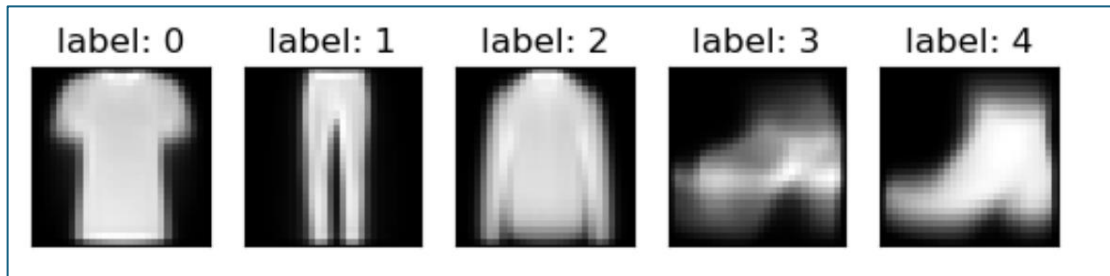


מטלה מסכמת מבוא ללמידת מכונה

חלק א: QDA

שאלה 2, סעיף ג:

הצגת התמונות של הווקטורים הממוצעים (סדרת התוחלות של כל תיוג)



מה שאנחנו רואים בכל תמונה היא הצגה של הממוצע של כל התמונות ששייכות לתיוג הספציפי שלה.

בפועל יוצאת תמונה שאפשר להבין ממנה (חוץ מתיוג 3-סנדלים) די בבירור את העצם המדובר אבל הוא מעט מטושטש מכיוון וכל פיקסל הוא ממוצע של אותו הפיקסל בכל התמונות בעלות תיוג זהה.

ניתן לראות שבתיוג מספר 3-סנדלים קשה להבחין בדיוק במה מדובר מבלי לראות תמונה ספציפית מאחר ויש הרבה סוגים שונים של סנדלים שגורמים לממוצע לא אחיד ו"מטושטש". בגלל שהממוצעים של תיוגים 3 ו-4 דומים זה לזה, נוכל להסיק שיהיה למסווג קשה יותר להבדיל ביניהם ובכך להוריד את רמת הדיוק.

שאלה 3:

```
Y_pred = classify(X_train, mu_ML, sigma_squared_ML, pi_ML)
print("The accuracy of the classifier for the train data is: " + str(accuracy(Y_train, Y_pred)))
```

The accuracy of the classifier for the train data is: 0.987875

רמת הדיוק של המסווג לסדרת האימון היא טובה מאוד (מעל 98%)
התבוננות על מטריצת הערבול מאשרת שיש מספר טעויות בסיווג של מגפיים שסווגו כסנדלים אך יש גם מספר של חולצות שסווגו כמכנסיים או כמעיל.

Confusion Matrix for train data

	T-shirt/top	Trouser	Coat	Sandal	Ankle boot
T-shirt/top	4653	54	86	0	0
Trouser	12	4808	10	0	0
Coat	4	10	4810	0	0
Sandal	4	0	0	4760	8
Ankle boot	0	0	0	103	4678
Predicted	T-shirt/top	Trouser	Coat	Sandal	Ankle boot

תמונה מ-ChatGPT: קישור לפרומפט

- **Precision:** This measures the accuracy of positive predictions. It is the ratio of true positive predictions to the total number of positive predictions made by the model. In other words, it answers the question: Of all the instances that the model predicted as positive, how many were actually positive?

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$

- **Recall (or Sensitivity):** This measures the model's ability to identify all relevant instances. It is the ratio of true positive predictions to the total number of actual positives. It answers the question: Of all the actual positive instances, how many did the model successfully identify?

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

Precision and recall for train data

Label T-shirt/top:

Precision: 0.9957200941579285, Recall: 0.9707907364907157

Label Trouser:

Precision: 0.986863711001642, Recall: 0.9954451345755694

Label Coat:

Precision: 0.9804321239298818, Recall: 0.9970978441127695

Label Sandal:

Precision: 0.9788196586469258, Recall: 0.9974853310980721

Label Ankle boot:

Precision: 0.9982927870251814, Recall: 0.9784563898765949

אנחנו נוכל לראות בהתבוננות בתוצאות ה-recall וה-precision שזה מאשר את מה שהסקנו בהתבוננות במטריצת הערבול.
ניתן לראות שה-recall וה-precision מתארים בצורה טובה את מה שראינו במטריצת הערבול, לחולצות ומגפיים ישנו recall נמוך יותר משאר התייגים מכיוון ויש מספר גבוהה יותר מהם שלא סווגו נכון.
בנוסף, בהתייחסות ל-precision נוכל לראות שהתייגים שבהם ערך ה-precision היה נמוך יותר הם התייגים שאליהם סווגו הטעויות בחולצה ומגפיים (חולצה למכנסיים ומעיל, ומגפיים לסנדלים).
מספר גבוהה של false positive בתיוג אחד גורם בצורה ישירה לעלייה של מספר ה-false negative בתיוגים אחרים וליחס הפוך בין ה-precision ל-recall שלהם.

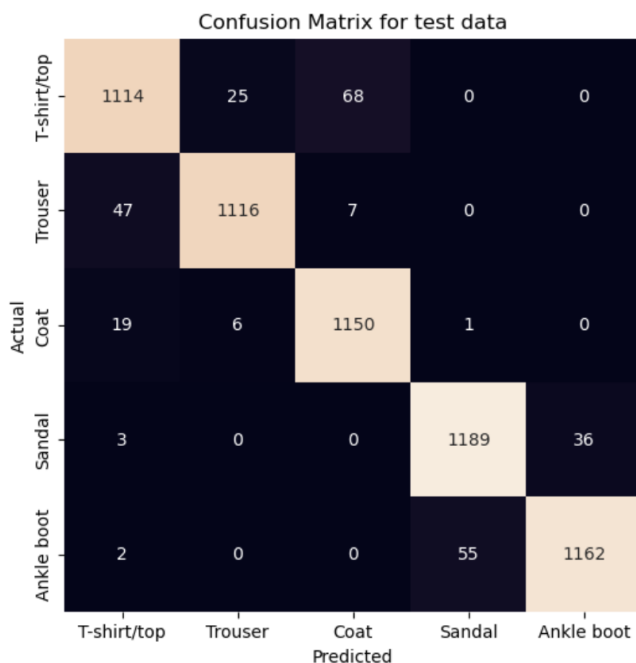
שאלה 4 – חזרה על שאלה 3 עם סדרת המבחן:

```
Y_pred = classify(X_test, mu_ML, sigma_squared_ML, pi_ML)
print("The accuracy of the classifier for the test data is: " + str(accuracy(Y_test, Y_pred)))
```

The accuracy of the classifier for the test data is: 0.9551666666666667

רמת הדיוק לסדרת המבחן גם היא גבוהה מאוד אם כי יותר נמוכה מדיוק סדרת האימון. התבוננות במטריצת הערבול מגלה לנו דברים שלא היו קיימים בסדרת המבחן כגון מספר גדול יותר של מכנסיים שסווגו כחולצות וגם סנדלים שסווגו כמגפיים. התיוג ה"בעייתי" ביותר הוא חולצה שזוכה לערכים יחסית נמוכים של גם recall וגם precision בהשוואה לשאר התיוגים מאחר ויש הרבה חולצות שלא תוייגו כראוי ויש מספר רב יחסית של מכנסיים שתוייגו כחולצה. ערך ה-precision הנמוך ביותר שייך למעיל מכיוון שיש מספר גדול של חולצות שתוייגו כמעיל.

שאר התוצאות דומות למה שראינו באימון למעט מספר גדול יותר של סנדלים שתוייגו בשגיאה כמגפיים, שזה מה שחזינו בהתחלה שיקרה.



Precision and recall for test data

Label T-shirt/top:

Precision: 0.940084388185654, Recall: 0.9229494614747308

Label Trouser:

Precision: 0.972972972972973, Recall: 0.9538461538461539

Label Coat:

Precision: 0.9387755102040817, Recall: 0.9778911564625851

Label Sandal:

Precision: 0.9550200803212852, Recall: 0.9682410423452769

Label Ankle boot:

Precision: 0.9699499165275459, Recall: 0.9532403609515997

חלק ב: LLM

שאלה 5:

ראשית, בחנו את ביצועי כל המסווגים הנתונים על ידי שימוש ב-cross validation ובדיקה של דיוק המסווג הממוצע על סדרת המבחן וסדרת האימון ושרטוט גרף במקרה ויש hyperparameters שניתן לשנות.
על פי תוצאות הניסויים שלנו, בחרנו ב-LDA שלפי דעתנו ייתן תוצאות מיטביות לסיווגים עתידיים של ה-LLM הנתון.

LDA

Cross validation train accuracy for LDA is: 0.7937275152574206
Cross validation test accuracy for LDA is: 0.7802632070746713

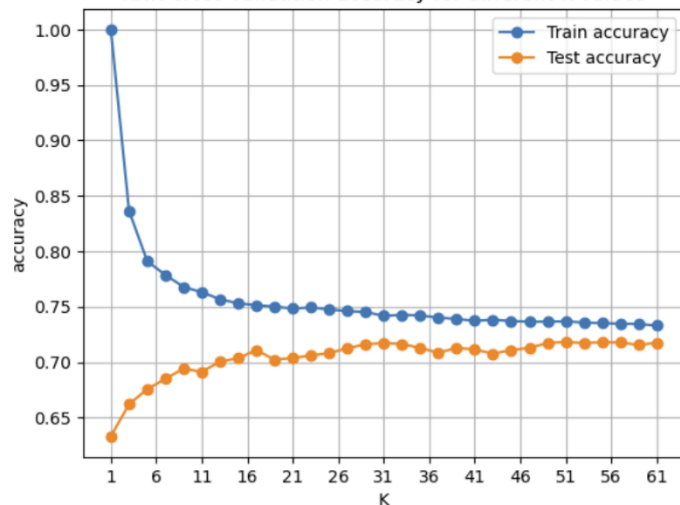
QDA

Cross validation train accuracy for QDA is: 0.8475678672842445
Cross validation test accuracy for QDA is: 0.7269725579900278

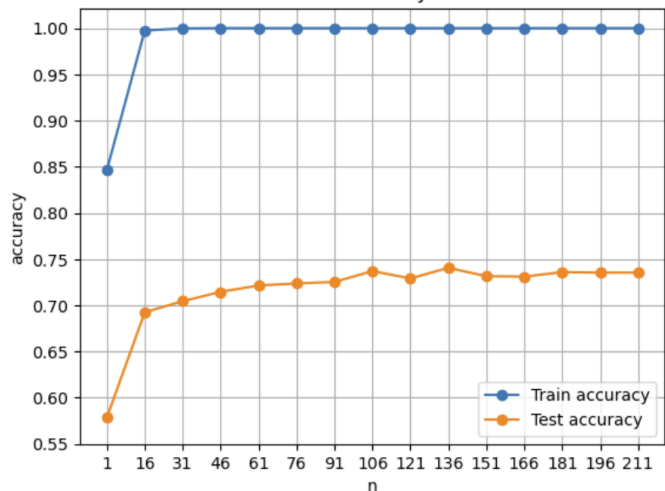
GNB

Cross validation train accuracy for GNB is: 0.6952714598023676
Cross validation test accuracy for GNB is: 0.6881635307573278

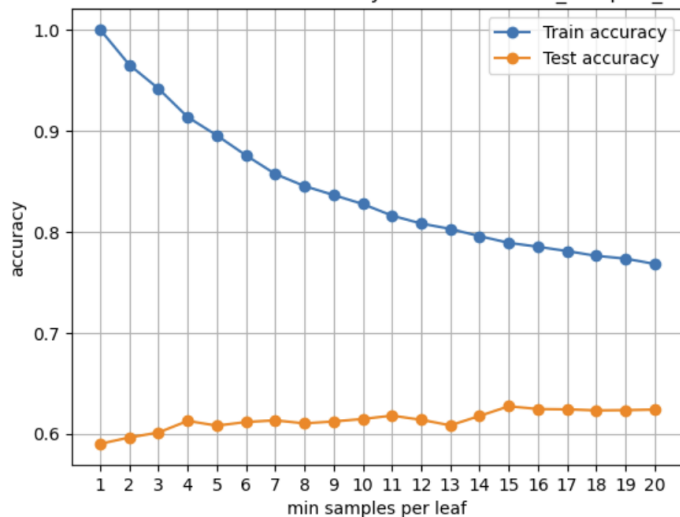
KNN cross validation accuracy for different K values



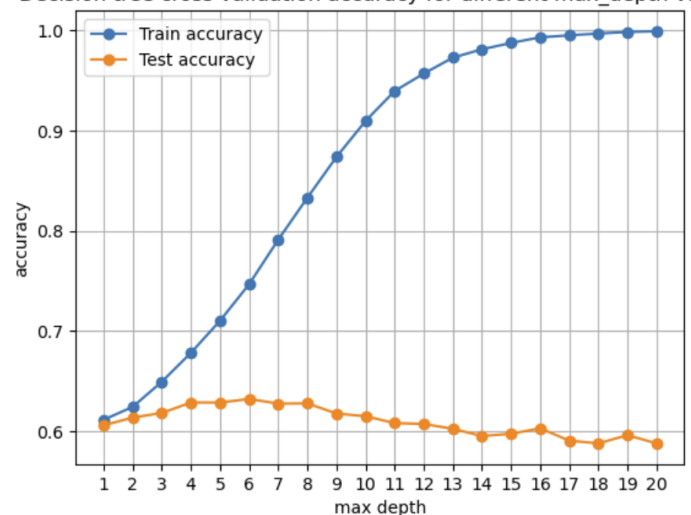
Random forest cross validation accuracy for different number of trees



Decision tree cross validation accuracy for different min_samples_leaf values



Decision tree cross validation accuracy for different max_depth values



בהתחלה חשבנו ש-Random forest ייתן לנו תוצאות טובות אך לאחר בדיקה ראינו שגם decision tree וגם random forest סובלים מ-Overfitting לסוג הדאטה שמודל ה-LLM הנתון מייצר. ל-Random forest וגם ל-decision tree מהר מאוד סיווג האימון הגיע ל-1 או קרוב ל-1 מה שמצביע על הבעיה בזמן שדיוק המבחן היה נמוך בהרבה.

לכאורה נראה בהתחלה ש-QDA ייתן את התוצאות המיטביות מאחר ודיוק סדרת האימון היה גבוהה, אך בבדיקה על סדרת המבחן קיבלנו תוצאות נמוכות יותר משמעותית מה שכןראה מצביע על ייתכנות של overfitting, כך שגם הוא לא אופטימלי לפתרון הבעיה.

מסווג ה-KNN קיבל תוצאות יחסית טובות וההבדל יחסית קטן בין דיוק המבחן והאימון כך שהוא לא סובל מ-overfitting אך עדיין התוצאות לא גבוהות מספיק כדי להיבחר.

מסווג ה-GNB גם הוא לא סובל מ-overfitting אך ההנחה שלו שהמאפיינים אינם תלויים זה בזה היא שגויה והוא מייצר דיוק נמוך יחסית גם לסדרת האימון וגם לסדרת המבחן.

לבסוף, ניתן לראות שמסווג ה-LDA בהתחלה לא נותן את התוצאות הטובות ביותר לאימון אך תוצאות המבחן הן הגבוהות ביותר מבין כל המסווגים שבחנו מה שמעיד על כך שהמשקלים שמייצר ה-LLM הם בעלי פיזור גאוסי והתייגים ניתנים להפרדה יחסית בקלות עם מסווג לינארי כמו LDA.

תוצאות האימון והמבחן קרובות מאוד זו לזו מה ששולל overfitting. כל אלו גורמים לנו להסיק שסוג הדאטה שמייצר ה-LLM הנתון מתאים להנחות היסוד של המסווג ודוגמאות עתידיות יסווגו בצורה האופטימלית בהשוואה לכל שאר המסווגים הנ"ל.

שאלה 6:

בחנו שוב את כל המסווגים כמו בשאלה 5 וראינו עלייה משמעותית באחוזי ההצלחה של כל המסווגים הנתונים.
בניגוד לשאלה קודמת, LDA לא בהכרח נותן את התוצאות הטובות ביותר אבל אנחנו עדיין סבורים שיש להשתמש ב-LDA לסיווגים עתידיים גם של מודל ה-LLM החדש.

LDA

Cross validation train accuracy for LDA is: 0.9061214740843215

Cross validation test accuracy for LDA is: 0.9010588833322121

QDA

Cross validation train accuracy for QDA is: 0.922382808634888

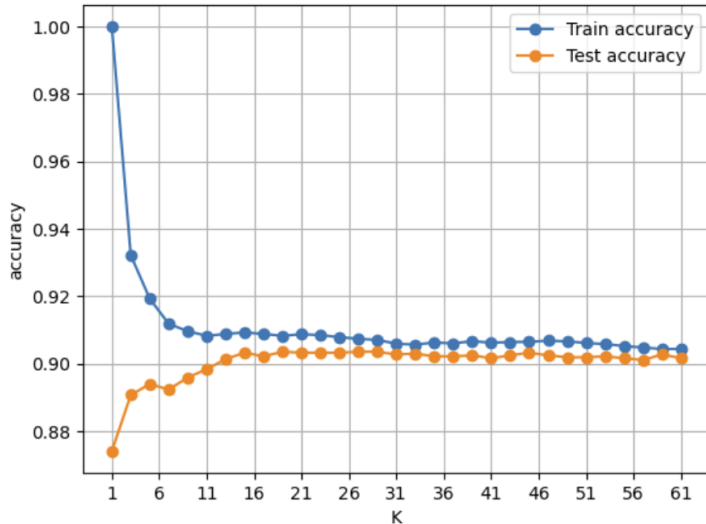
Cross validation test accuracy for QDA is: 0.9032454232169421

GNB

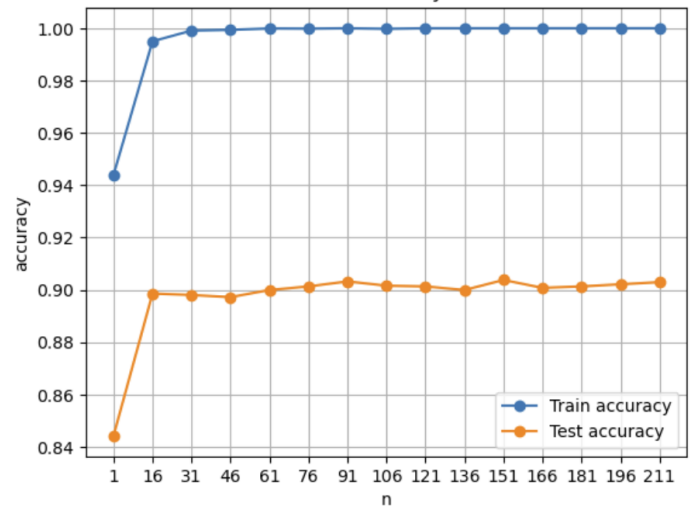
Cross validation train accuracy for GNB is: 0.8976492650861305

Cross validation test accuracy for GNB is: 0.8966865510977552

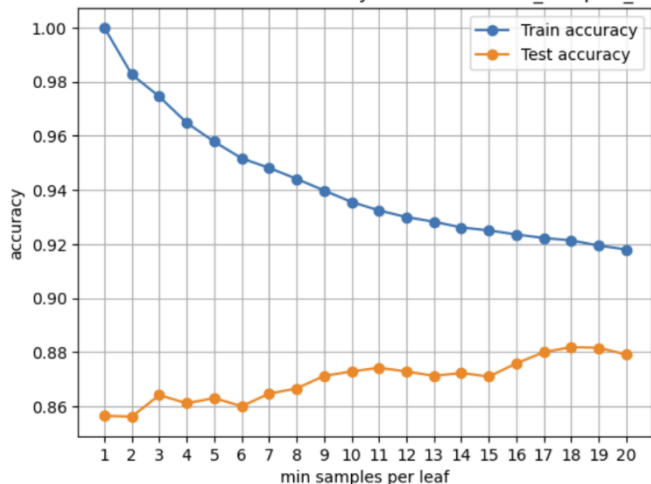
KNN cross validation accuracy for different K values



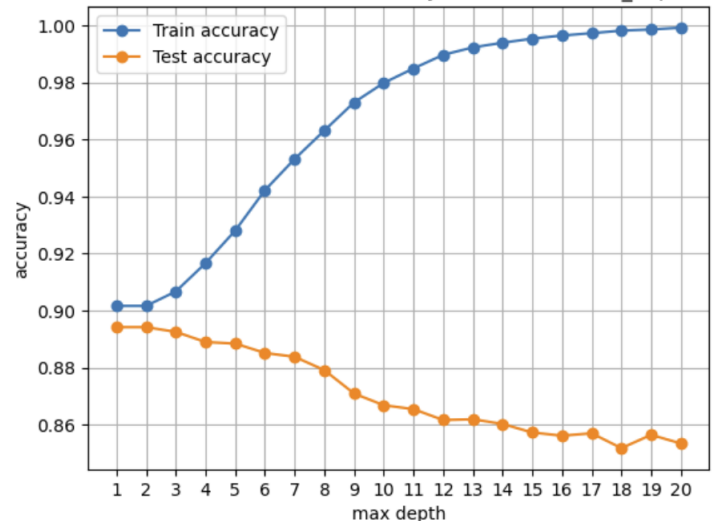
Random forest cross validation accuracy for different number of trees



Decision tree cross validation accuracy for different min_samples_leaf values



Decision tree cross validation accuracy for different max_depth values



בדומה לשאלה 5, Random forest ו-Decision tree סובלים מ-overfitting יחסית מהר בסוג הדאטה של מודל ה-LLM הנתון, אחוזי הדיוק לסדרת המבחן לא מרשימים ביחס למסווגים האחרים ודיוקי סדרת האימון גבוהים מדי ורחוקים מדי מדיוקי המבחן.

KNN, QDA ו-LDA נותנים את התוצאות הטובות ביותר וכמעט זהות לסדרת המבחן ואינם סובלים מ-overfitting ולכן נבחר מאחד מהם.

KNN נותן תוצאות טובות מאוד, טובות בהרבה מבשאלה הקודמת מה שמעיד שהמשקלים ל-tokens בעלי משמעות חיובית (או שלילית) מקובצים קרוב יחד ולמודל קל לסווג אותם כמעט בכל ערכי ה-K שבדקנו.

בכל זאת לא בחרנו ב-KNN בגלל שלכל נקודה שנרצה לסווג יש עלות חישובית גבוהה יחסית מאחר וצריך לבצע את החישוב של כל המרחקים מ-K השכנים הקרובים בכל דוגמא מחדש בשונה מ-LDA, QDA, GNB שנדרשים חישובים רבים רק בשלב האימון.

QDA נותן תוצאות טובות בהרבה משאלה 5 וגם ניכר שאינו סובל מ-overfitting במידה שהיתה מקודם (למרות שנראה שעדיין מעט יותר מ-LDA), הסיבה שבכל זאת לא בחרנו במסווג זה היא מאחר ו-LDA פשוט יותר חישובית ועדיין מקנה דיוק גבוה וכמעט זהה.

GNB הראה גם הוא שיפור משמעותי בדיוק מה שמראה שהדאטה שהתקבלה ממודל ה-LLM החדש מתאימה יותר מהישן לסוג המסווג הזה אך עדיין אחוזי הדיוק נמוכים יותר מהמסווגים בעלי הדיוק הטוב ביותר ולכן זהו לא המסווג הטוב ביותר לבעיה.

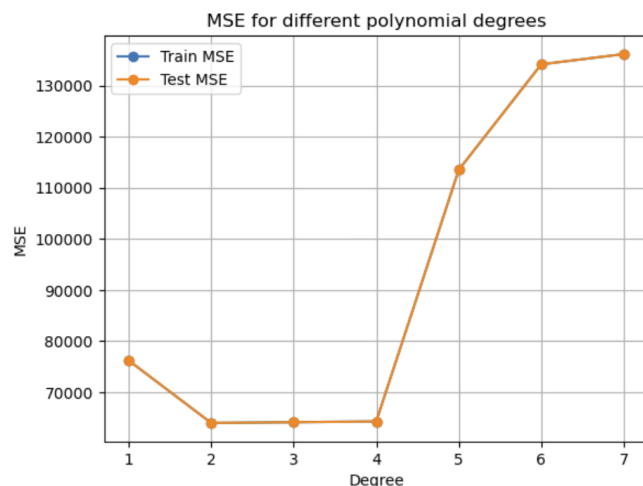
LDA מניב את התוצאות הטובות ביותר ביחד עם QDA ו-KNN וכל זאת עם 0 overfitting. לבסוף החלטנו לבחור את המסווג הזה מאחר והוא מתאים מאוד לסוג הדאטה שמתקבל ממודל ה-LLM לבעיית סיווג בינארית שכזאת שניתנת להפרדה בקלות בצורה לינארית, בזמן שעלות האימון שלו היא הנמוכה ביותר מבניהם.

חלק ג: רגרסיה

בהתחלה לאחר טעינת הדאטה, נאלצנו להתאים אותה שתהיה מיוצגת בצורה מספרית שמאפשרת אימון של המשערך.
בעמודת Date החלטנו להשאיר רק את השעה ולהמיר אותה לדקות (0-1440).
את כל העמודות שמכילות שמות של מיקומים בעיר המרנו ל-Map של מיקומים ייחודיים ממוספרים בסדר אלפבתי.
בנוסף גם שינינו את עמודת Type שיהיו ממופים לערכים מספריים.
יצרנו 2 משתנים שנקראים walk_trips ו-car_trips שכל אחד מהם הוא map שה-key הוא שם של מיקום וה-value הוא רשימה של כל המיקומים שניתן להגיע אליהם בצורה ישירה מהמיקום ההתחלתי (כפי שנאמר בשאלה אין נקודות באמצע). [פרומפט](#)

שאלות 7 ו-8:

ביצענו cross validation ובחינה של ה-MSE עבור דרגות שונות של המשערך ומצאנו שדרגה 4 מניבה את התוצאות האופטימליות וממזערת את ערך ה-MSE הממוצע.



לאחר כל המניפולציות שעשינו על הדאטה ומציאת הדרגה האופטימלית, אימנו את המשערך על מסלולים בהליכה כדי לקבל ווקטור משקלים w שנשתמש בו בהמשך לשערך נסיעות.

בנינו פונקציה שנקראת find_best_route_bfs שמקבלת את הגרף שלנו (walk/car trips), נקודת התחלה, נקודת סיום וזמן התחלה.

הפונקציה היא בעצם מימוש משודרג של אלגוריתם BFS (Breadth First Search) שבו אנחנו מתחילים מנקודת ההתחלה ומחפשים מסלול אל נקודת הסיום, בכל שלב של הדרך מתבצע שיערוך של זמן המעבר בין הנקודה הנוכחית אל הנקודה הבאה ומתווסף לזמן הכולל של המסלול.

כאשר האלגוריתם מגיע למסלול שהזמן שלו ארוך מהמסלול עם הזמן הטוב ביותר שנמצא עד כה הוא יזנח אותו ויעבור לבדיקת המסלול הבא.

מעצם היכולות של BFS, ייקח זמן קצר יותר למציאת מסלולים בין נקודות קרובות (פחות נקודות ביניים), ניסינו בהתחלה מימוש מבוסס DFS (Depth First Search) אך לפעמים זמני הריצה היו ארוכים מאוד למרות שהמסלול אמור להיות יחסית קצר (אם האלגוריתם החל להיכנס לעומק של מסלול שאינו אופטימלי למרות שיש מסלול חלופי קצר יותר). ולבסוף הפונקציה מחזירה את המסלול האופטימלי שהיא מצאה ואת הזמן בשניות.

[פרומפט ל-BFS](#), [פרומפט ל-DFS](#)

לאחר שקיבלנו את התוצאות עבור אנשים שמעוניינים להתנייד בהליכה, אימנו מחדש את המסווג על מסלולים של כלי רכב והרצנו מחדש את הפונקציה כדי למצוא את המסלולים האופטימליים עבור נסיעות ברכב לנתונים שניתנו בדאטת המבחן.

שאלה 9:

בנינו פונקציית עזר שמקבלת את מתי התחלנו לחכות לאוטובוס בתחנה מסויימת, את לוחות הזמנים של האוטובוס, והיא מחזירה את מספר השניות שחיכינו מהרגע שהגענו לתחנה ועד שהגענו ליעד (מחזיר inf אם לא קיים מסלול).

ביצענו שינוי בפונקציה הראשית משאלות 7 ו-8 כך שיתבצע בכל שלב שערור של זמן הנסיעה/הליכה ונבחר את הזמן הקצר ביותר לכל קטע דרך ולבסוף מוחזרים המסלול האופטימלי שמצאנו, מה הדרך שבה לקחנו בכל שלב (הליכה/רכב/אוטובוס) וכמה זמן לקח המסלול בדרך שנמצאה.